

**COMPSYS302**

**Design: Software Practice**

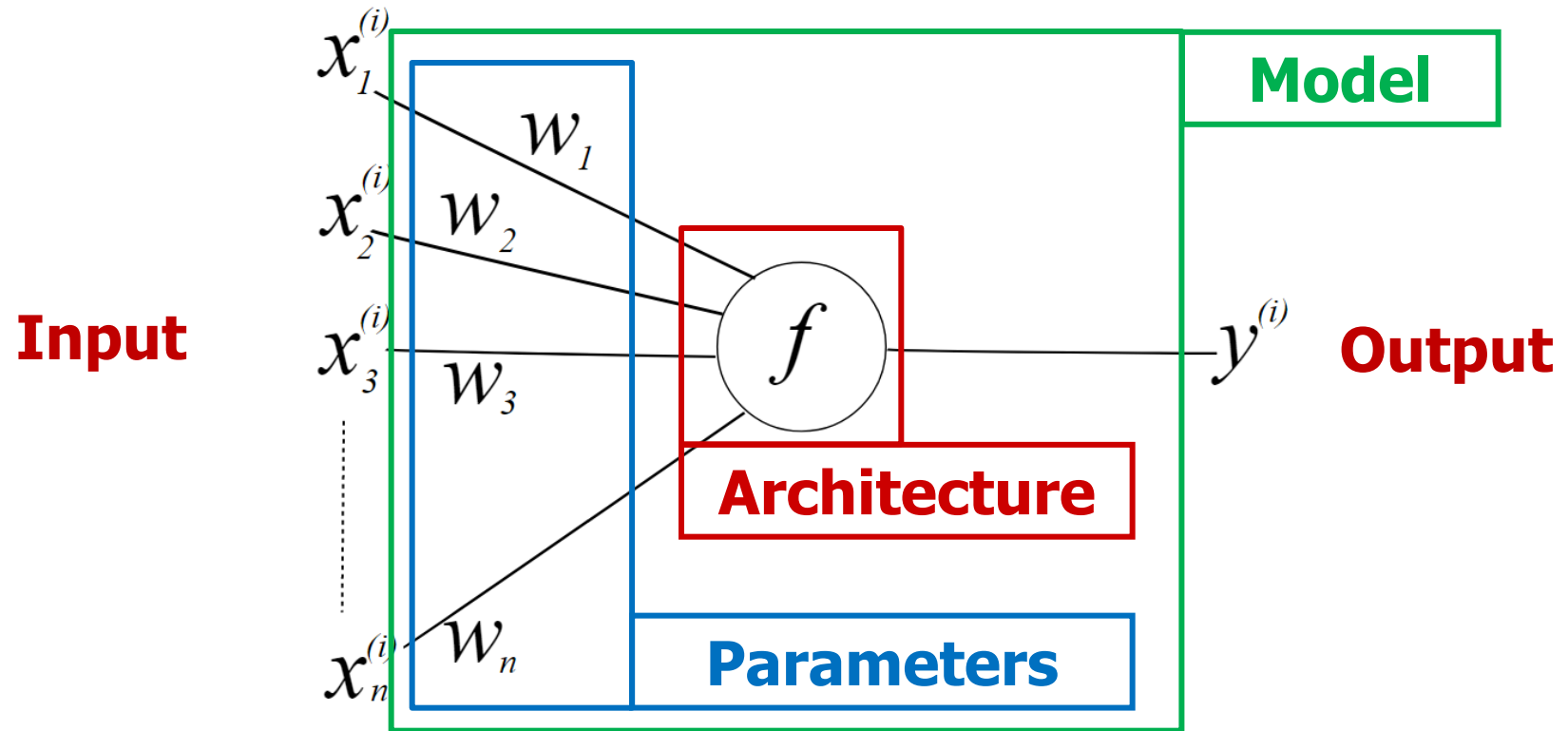
**Introduction to AI #4 – More CNN**

**Ho Seok AHN**

hs.ahn@auckland.ac.nz

# 1. Neural Network

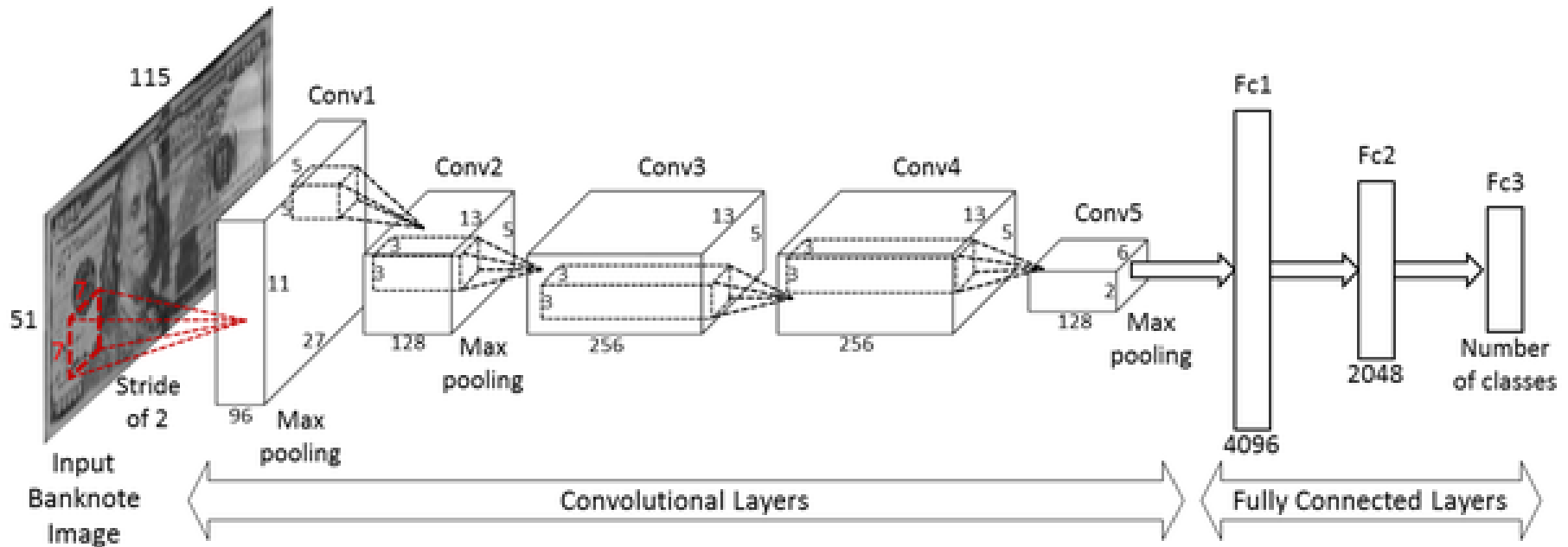
## □ What is Model?



**Model = Architecture + Parameters**

## 2. Concept of CNN

### □ CNN



- How deep is **convolutional layers**?
- **Size of kernel matrix** for convolutions?
- Which **polling** methods?
- Which **activation** functions?

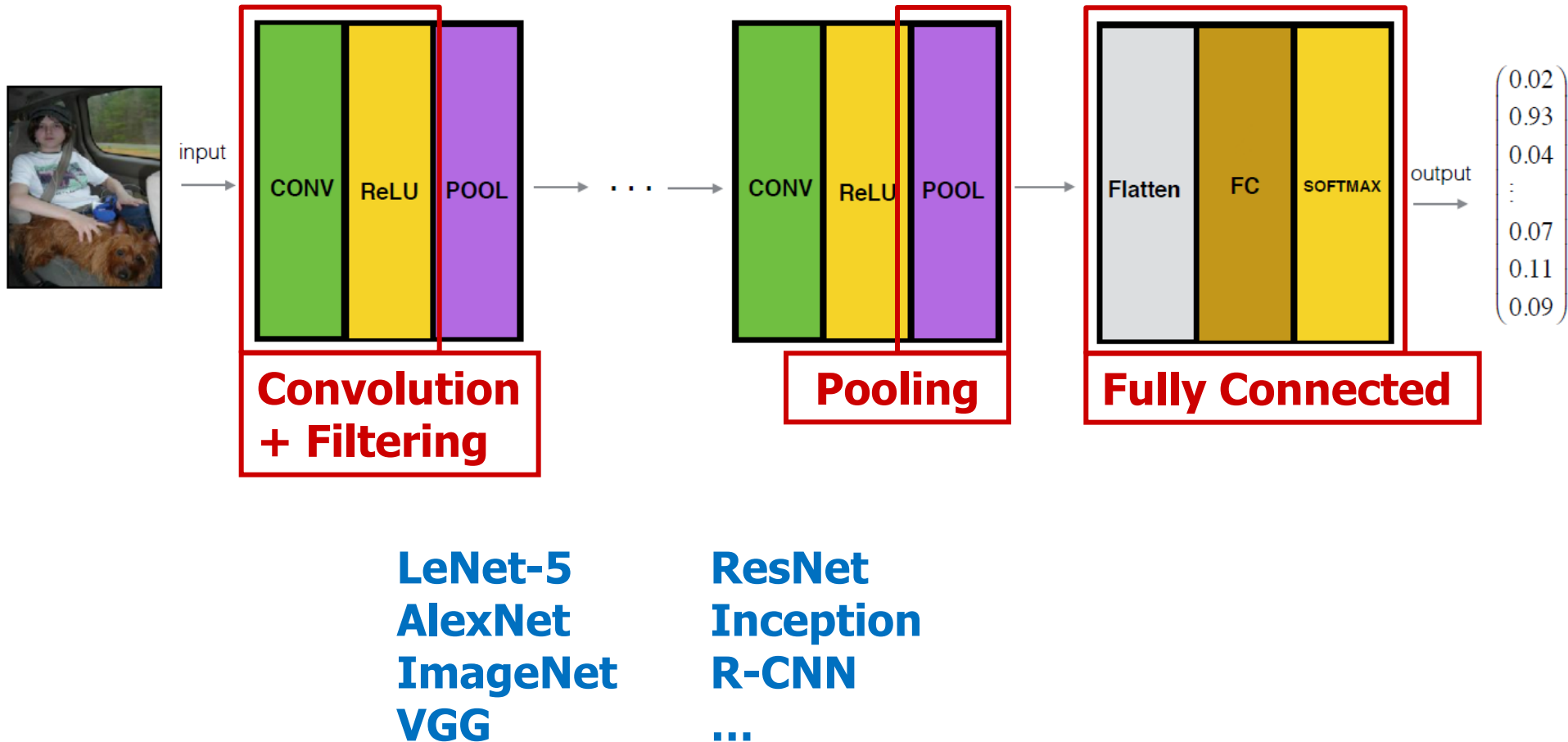
...



**Makes  
different  
models**

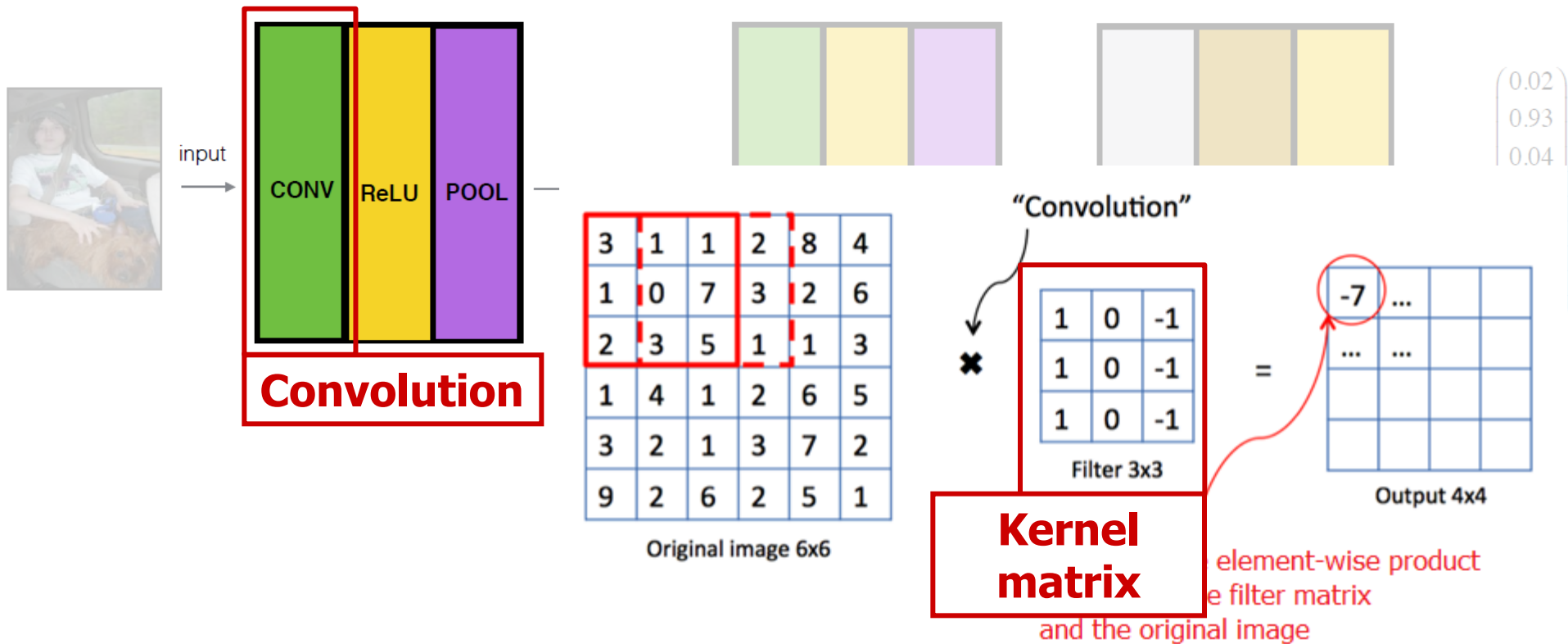
## 2. Concept of CNN

### □ Recap - CNN



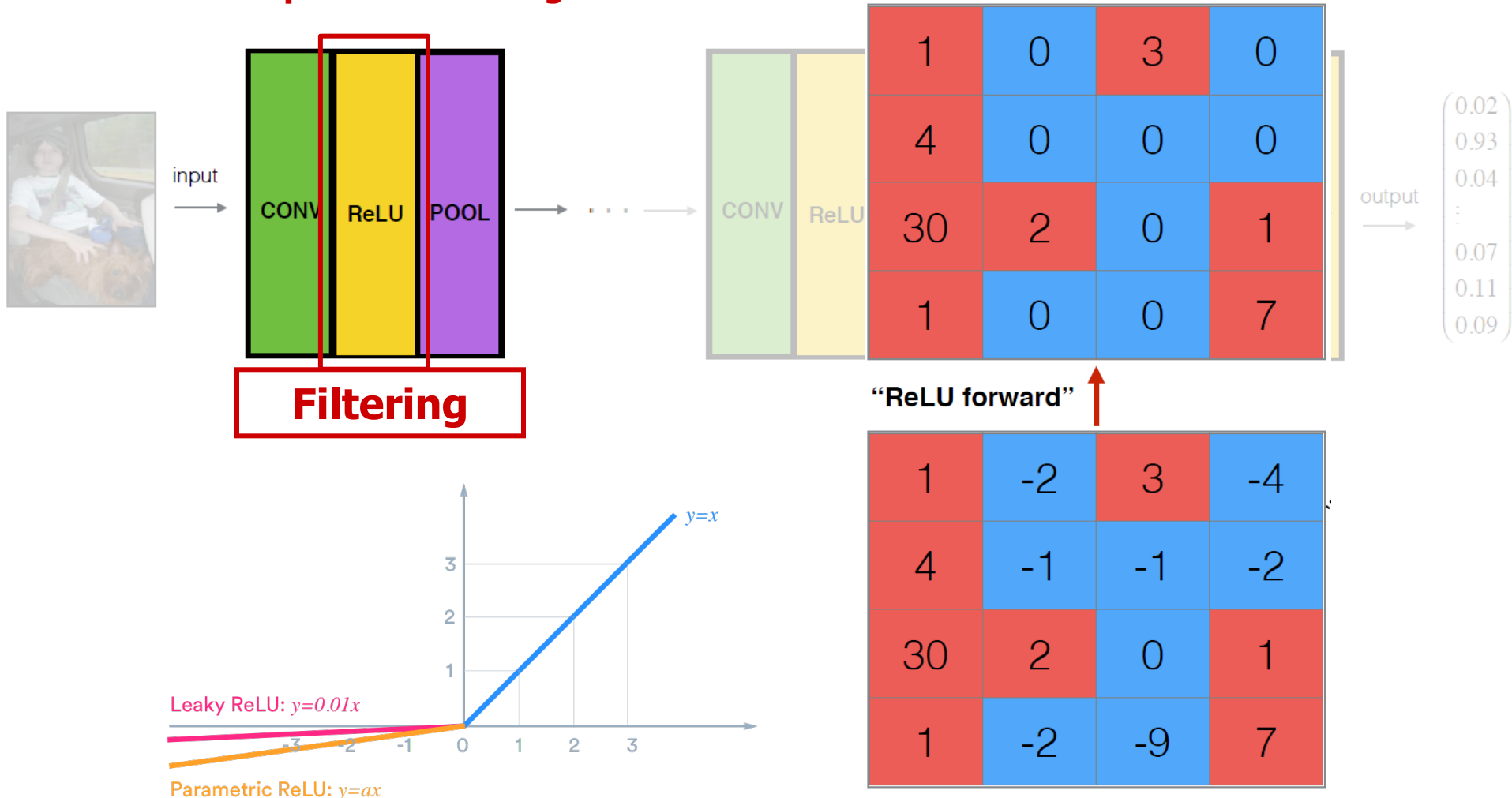
## 2. Concept of CNN

### □ Recap - CNN Convolution



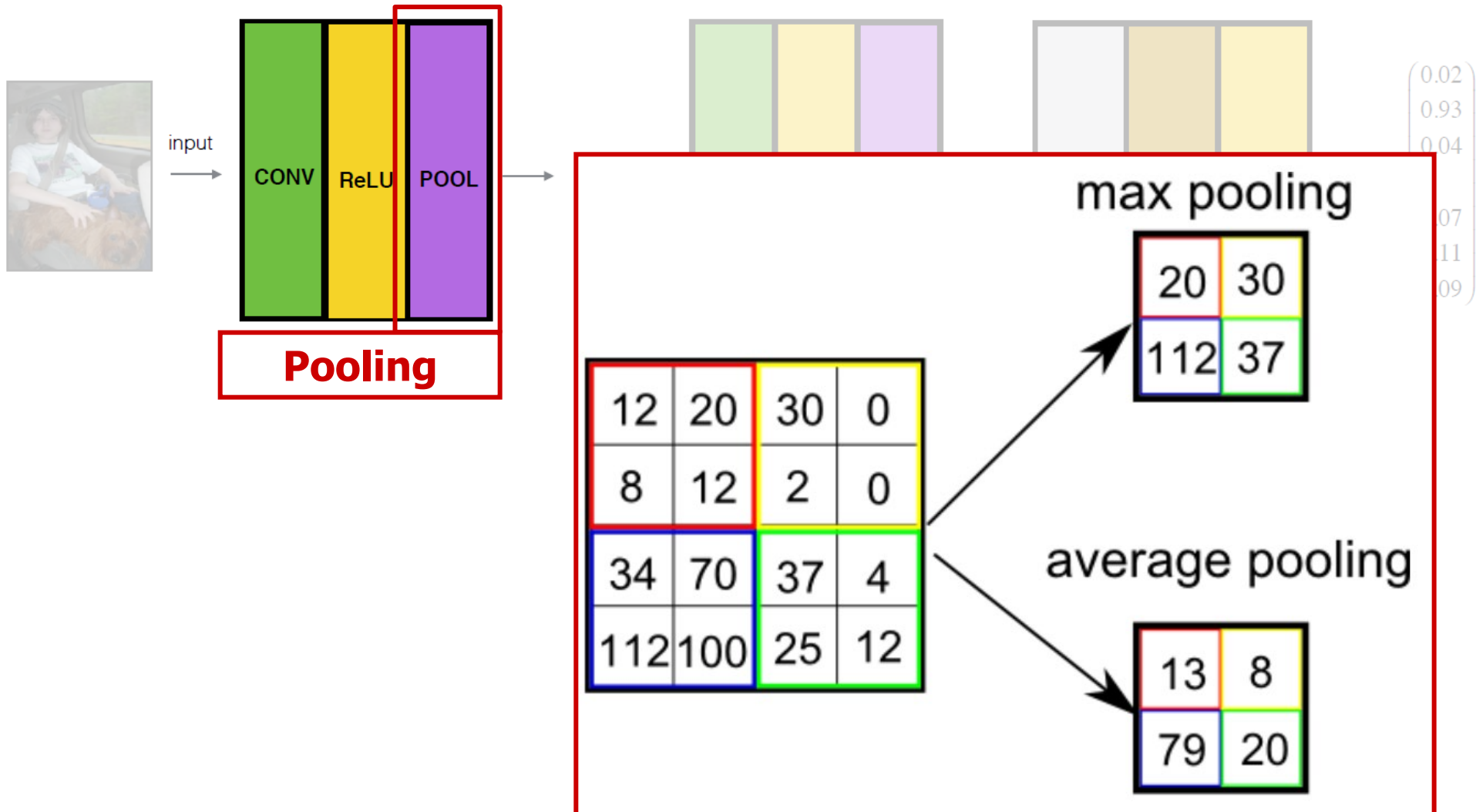
## 2. Concept of CNN

### □ Recap - CNN Filtering



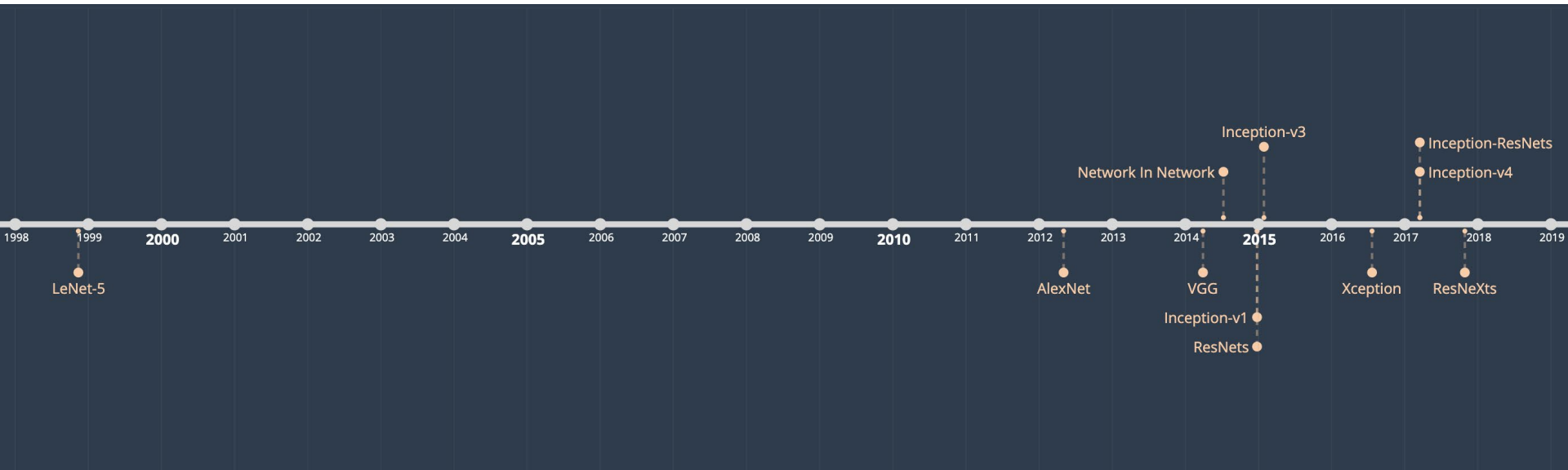
## 2. Concept of CNN

### □ Recap - CNN Polling



# 3. Different models for CNN

## □ CNN



**Simple models**

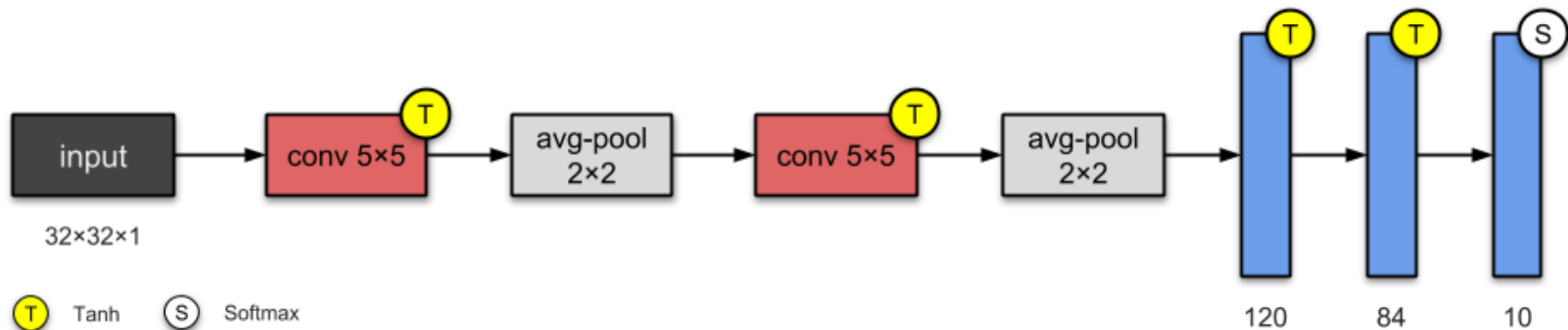


**Complex models**

# 3. Different models for CNN

## □ LeNet-5 (1998)

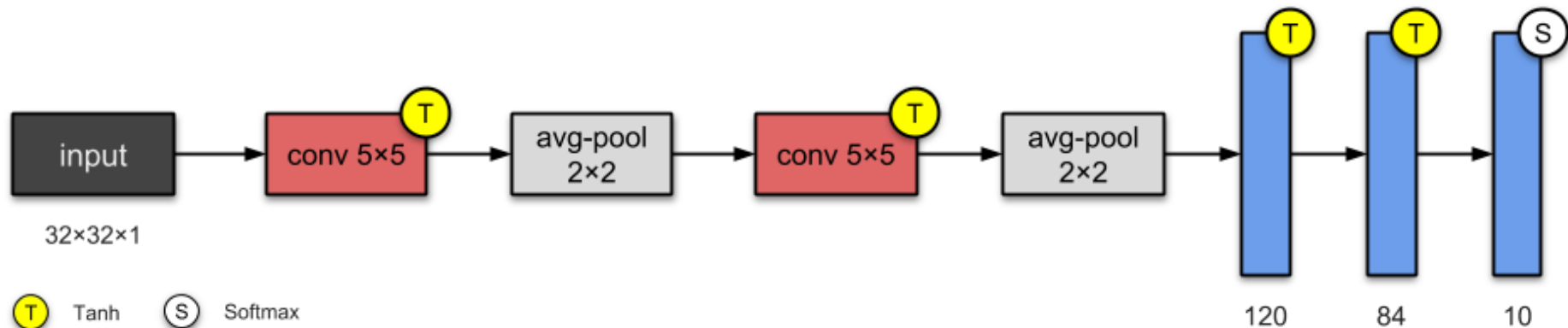
- one of the **simplest** architectures.
  - has 3 convolutional and 2 sub-sampling layers. → named as “5”
  - has about 60K parameters.
  - The average-pooling layer as we know it now was called a “sub-sampling layer”.
- This architecture has become the **standard ‘template’**: **stacking convolutions and pooling layers, and ending the network with one or more fully-connected layers.**



# 3. Different models for CNN

## □ LeNet-5 (1998)

| Layer  |                 | Feature Map | Size  | Kernel Size | Stride | Activation |
|--------|-----------------|-------------|-------|-------------|--------|------------|
| Input  | Image           | 1           | 32x32 | -           | -      | -          |
| 1      | Convolution     | 6           | 28x28 | 5x5         | 1      | tanh       |
| 2      | Average Pooling | 6           | 14x14 | 2x2         | 2      | tanh       |
| 3      | Convolution     | 16          | 10x10 | 5x5         | 1      | tanh       |
| 4      | Average Pooling | 16          | 5x5   | 2x2         | 2      | tanh       |
| 5      | Convolution     | 120         | 1x1   | 5x5         | 1      | tanh       |
| 6      | FC              | -           | 84    | -           | -      | tanh       |
| Output | FC              | -           | 10    | -           | -      | softmax    |



# 3. Different models for CNN

## □ LeNet-5 (1998)

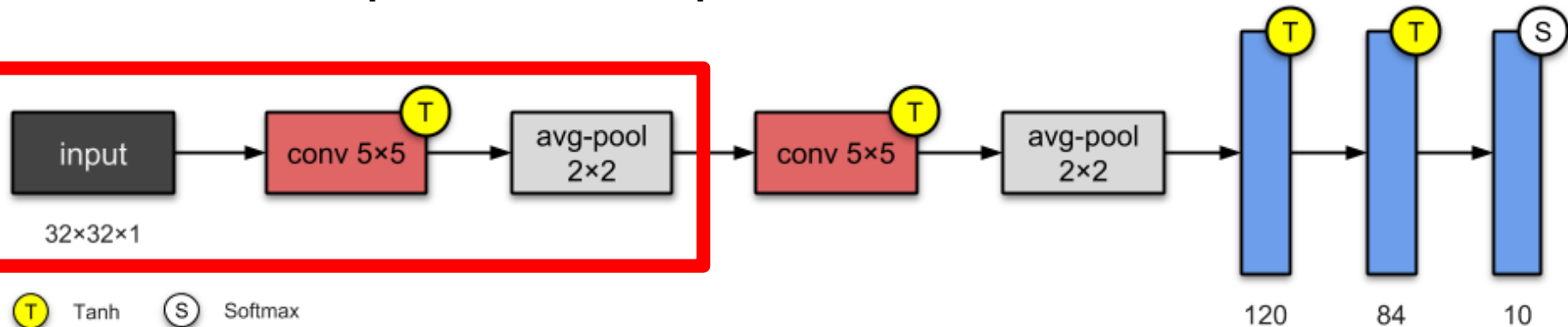
| Layer |                 | Feature Map | Size  | Kernel Size | Stride | Activation |
|-------|-----------------|-------------|-------|-------------|--------|------------|
| Input | Image           | 1           | 32x32 | -           | -      | -          |
| 1     | Convolution     | 6           | 28x28 | 5x5         | 1      | tanh       |
| 2     | Average Pooling | 6           | 14x14 | 2x2         | 2      | tanh       |

□ **Conv1**: 6 feature maps (i.e. Edge detection (horizontal, vertical, diagonal), Blob detection),  
5×5 kernels, stride 1

□ **Input**: 32×32 grayscale images → **Output**: 28×28×6

□ **Avg Pooling1**: 2×2 kernels, stride 2

□ **Input**: 28×28×6 → **Output**: 14×14×6



# 3. Different models for CNN

## □ LeNet-5 (1998)

|   | Layer           | Feature Map | Size  | Kernel Size | Stride | Activation |
|---|-----------------|-------------|-------|-------------|--------|------------|
| 3 | Convolution     | 16          | 10x10 | 5x5         | 1      | tanh       |
| 4 | Average Pooling | 16          | 5x5   | 2x2         | 2      | tanh       |

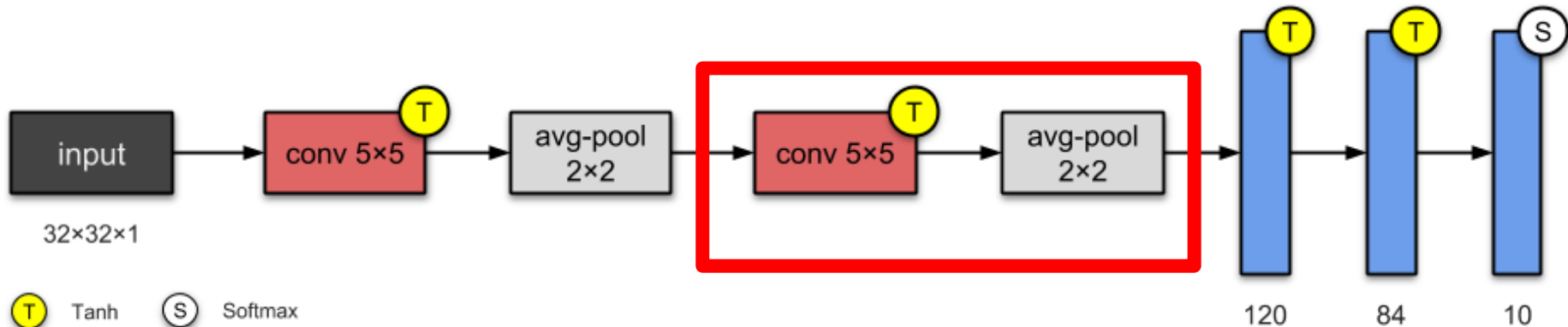
□ **Conv2:** 16 feature maps,  $5 \times 5$  kernels, stride 1

□ **Input:**  $14 \times 14 \times 6 \rightarrow$  **Output:**  $10 \times 10 \times 16$

□ **Avg Pooling2:**  $2 \times 2$  kernels, stride 2

□ **Input:**  $10 \times 10 \times 16 \rightarrow$  **Output:**  $5 \times 5 \times 16$

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|
| 0 | X |   |   |   | X | X | X |   |   | X | X  | X  | X  |    | X  | X  |
| 1 | X | X |   |   |   | X | X | X |   |   | X  | X  | X  | X  |    | X  |
| 2 | X | X | X |   |   |   | X | X | X |   |    | X  |    | X  | X  | X  |
| 3 |   | X | X | X |   |   | X | X | X | X |    |    | X  |    | X  | X  |
| 4 |   |   | X | X | X |   |   | X | X | X | X  |    | X  | X  |    | X  |
| 5 |   |   |   | X | X | X |   |   | X | X | X  | X  |    | X  | X  | X  |



# 3. Different models for CNN

## □ LeNet-5 (1998)

|        | Layer       | Feature Map | Size | Kernel Size | Stride | Activation |
|--------|-------------|-------------|------|-------------|--------|------------|
| 5      | Convolution | 120         | 1x1  | 5x5         | 1      | tanh       |
| 6      | FC          | -           | 84   | -           | -      | tanh       |
| Output | FC          | -           | 10   | -           | -      | softmax    |

□ **Conv3**: 120 feature maps (i.e. Intersection patterns),  $5 \times 5 \times 16$  kernels, stride 1

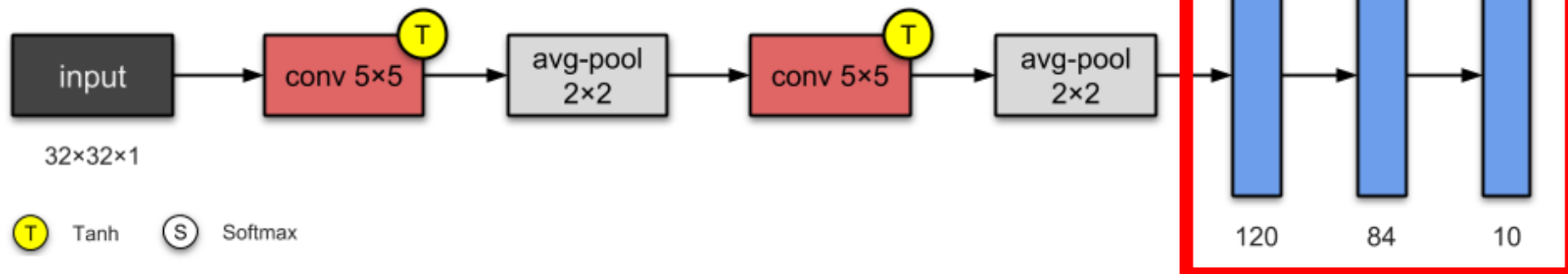
□ **Input**:  $5 \times 5 \times 16 \rightarrow$  **Output**:  $1 \times 1 \times 120$

□ **FC1**: Feedforward NN with 84 units

□ **Input**:  $1 \times 1 \times 120 \rightarrow$  **Output**: 84

□ **FC2**: Feedforward NN with 10 units

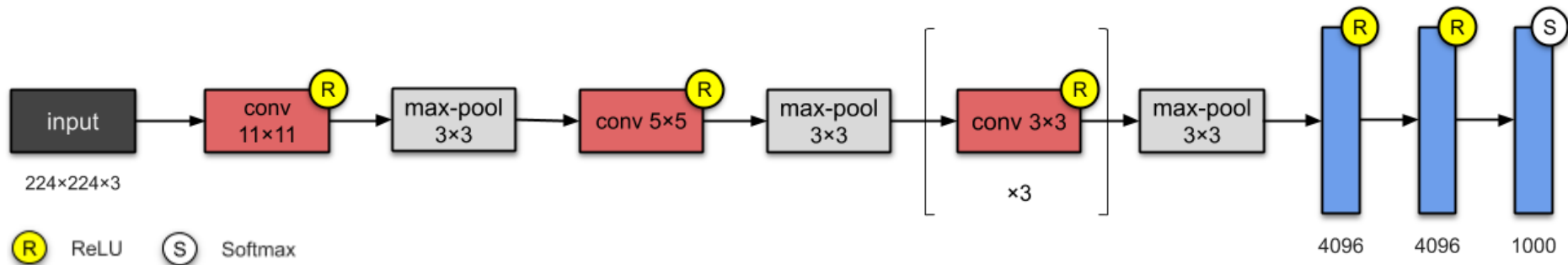
□ **Input**: 84  $\rightarrow$  **Output**: 10



# 3. Different models for CNN

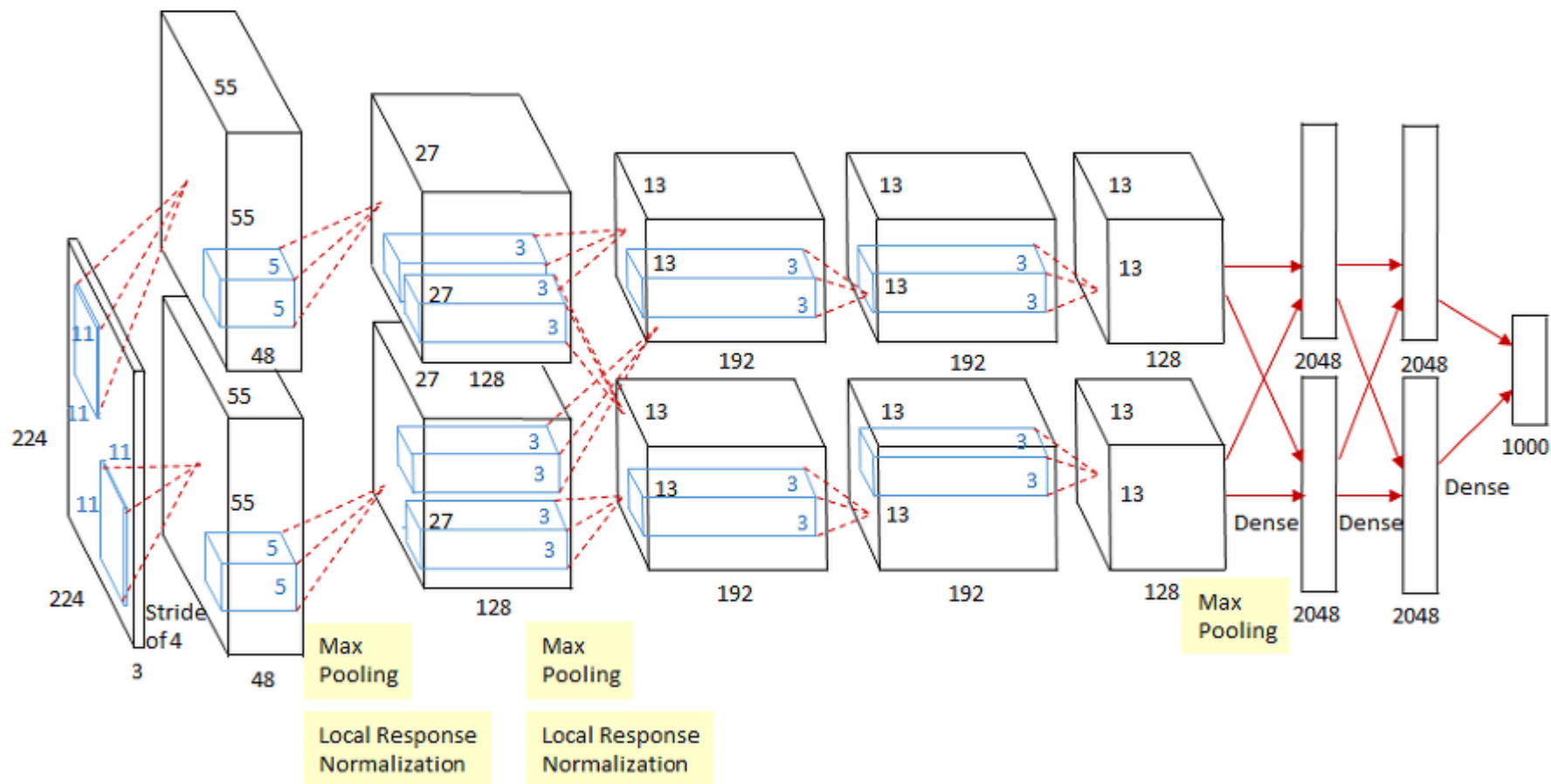
## □ AlexNet (2012)

- Original paper: "ImageNet Classification with Deep Convolutional Neural Networks"
- First author of this paper is "Alex Krizhevsky" → named as "AlexNet"
- 1<sup>st</sup> in the "ImageNet Large Scale Visual Recognition Challenge" (2012)
  - VGG: 2<sup>nd</sup> (2014)
  - ResNet: 1<sup>st</sup> (2015)
- Parallel structure for using 2 x GPUs



# 3. Different models for CNN

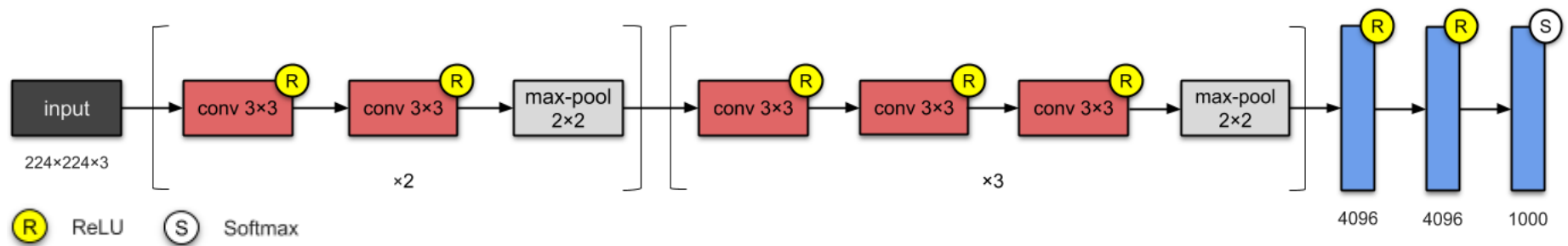
## □ AlexNet (2012)



# 3. Different models for CNN

## □ VGG-16 (2014)

- invented by VGG(Visual Geometry Group) group, University of Oxford, UK
  - **stacks more layers** onto AlexNet, and **use smaller size filters** ( $2 \times 2$  and  $3 \times 3$ ).
  - has 16 layers: 13 convolutional and 3 fully-connected layers.
  - has about 138M parameters.
  - They also designed a deeper variant, VGG-19.
- This architecture has deeper networks.



# 3. Different models for CNN

## □ VGG-16 (2014)

```
def __init__(self, num_classes=1000):
    super(VGG16, self).__init__()
    self.conv1_1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
    self.conv1_2 = nn.Conv2d(64, 64, kernel_size=3, padding=1)

    self.conv2_1 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
    self.conv2_2 = nn.Conv2d(128, 128, kernel_size=3, padding=1)

    self.conv3_1 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
    self.conv3_2 = nn.Conv2d(256, 256, kernel_size=3, padding=1)
    self.conv3_3 = nn.Conv2d(256, 256, kernel_size=3, padding=1)

    self.conv4_1 = nn.Conv2d(256, 512, kernel_size=3, padding=1)
    self.conv4_2 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
    self.conv4_3 = nn.Conv2d(512, 512, kernel_size=3, padding=1)

    self.conv5_1 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
    self.conv5_2 = nn.Conv2d(512, 512, kernel_size=3, padding=1)
    self.conv5_3 = nn.Conv2d(512, 512, kernel_size=3, padding=1)

    self.fc1 = nn.Linear(512*7*7, 4096)
    self.fc2 = nn.Linear(4096, 4096)
    self.fc3 = nn.Linear(4096, num_classes)

    self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
```

```
def forward(self, x):
    x = F.relu(self.conv1_1(x))
    x = F.relu(self.conv1_2(x))
    x = self.pool(x)

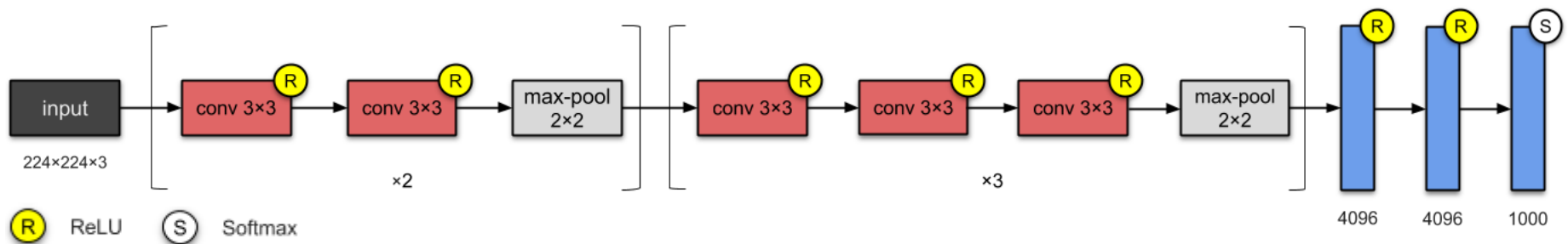
    x = F.relu(self.conv2_1(x))
    x = F.relu(self.conv2_2(x))
    x = self.pool(x)

    x = F.relu(self.conv3_1(x))
    x = F.relu(self.conv3_2(x))
    x = F.relu(self.conv3_3(x))
    x = self.pool(x)

    x = F.relu(self.conv4_1(x))
    x = F.relu(self.conv4_2(x))
    x = F.relu(self.conv4_3(x))
    x = self.pool(x)

    x = F.relu(self.conv5_1(x))
    x = F.relu(self.conv5_2(x))
    x = F.relu(self.conv5_3(x))
    x = self.pool(x)

    x = torch.flatten(x, 1)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
```



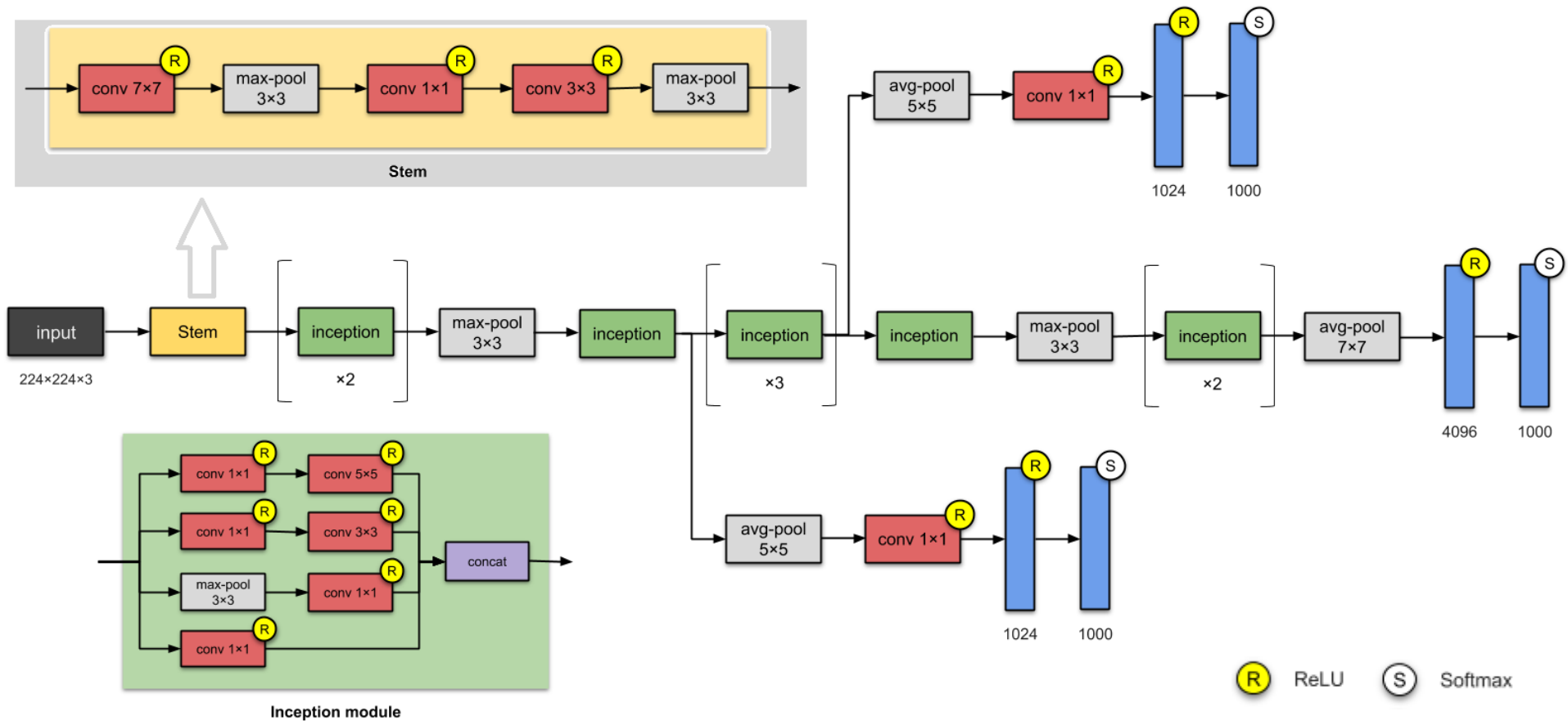
# 3. Different models for CNN

## □ Inception-v1 (2015)

- invented by **Google**, University of Michigan, University of North Carolina, USA
- has 22 layers: 14 convolutional and 6 fully-connected layers.
- has about 5M parameters.
- has **two auxiliary networks** to encourage discrimination in the lower stages of the classifier.
- **parallel towers of convolutions** with different filters, followed by concatenation, captures different features at  $1 \times 1$ ,  $3 \times 3$  and  $5 \times 5$ .
- $1 \times 1$  convolutions are used for dimensionality reduction to remove computational bottlenecks.
- ➔ Instead of stacking convolutional layers, this architecture **stacks modules or blocks**, within which are convolutional layers.

# 3. Different models for CNN

## □ Inception-v1 (2015)



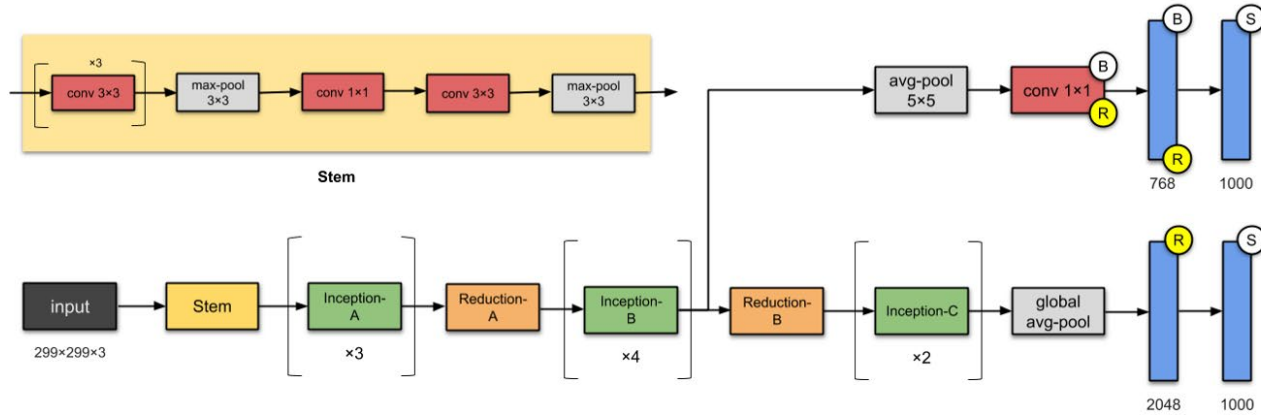
# 3. Different models for CNN

## □ Inception-v3 (2015)

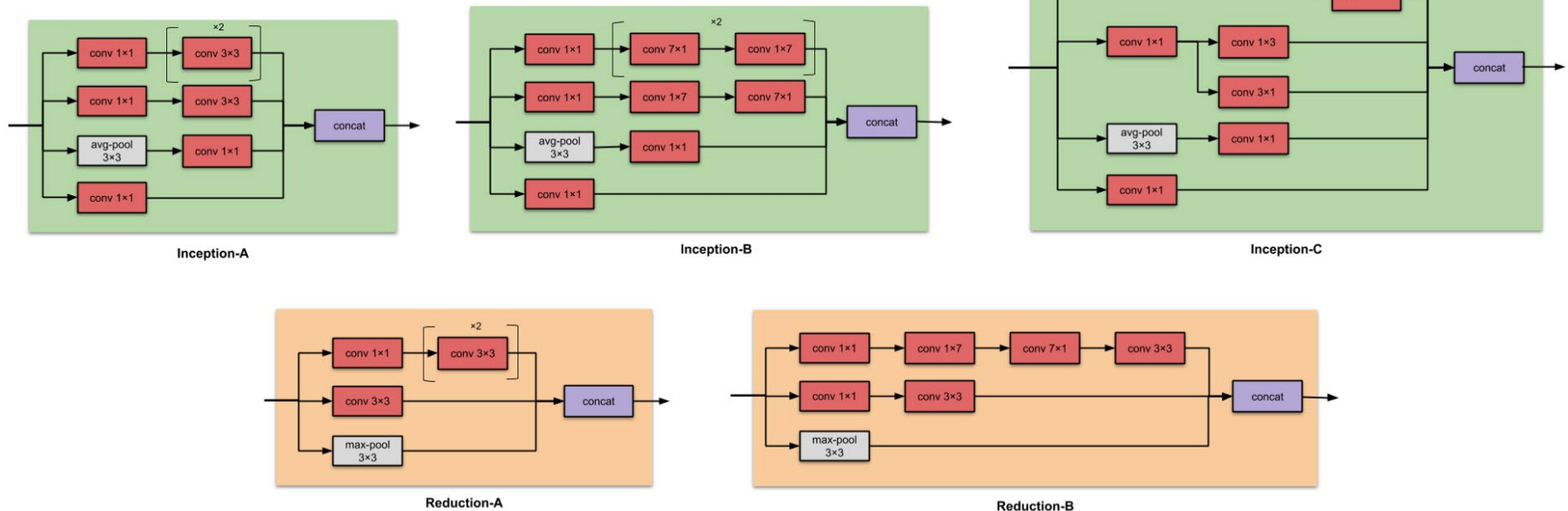
- invented by **Google**, University College London
  - is a **successor to Inception-v1**.
  - has about 24M parameters.
  - Inception-v2 is a prototype of v3, and similar to v3, so not commonly used.
  - avoid representational bottlenecks by drastically reducing the input dimensions of the next layer.
  - has more **efficient computations using factorising  $n \times n$  convolutions** into asymmetric convolutions:  $n \times 1$  and  $1 \times n$  convolutions.
- ➔ Instead of using the same module, this architecture **stacks different modules or blocks**.

# 3. Different models for CNN

## □ Inception-v3 (2015)



R ReLU   
 S Softmax   
 B Batch normalisation



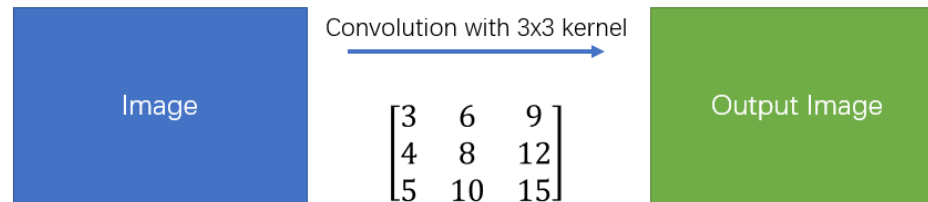
# 3. Different models for CNN

## □ Spatial separable convolutions

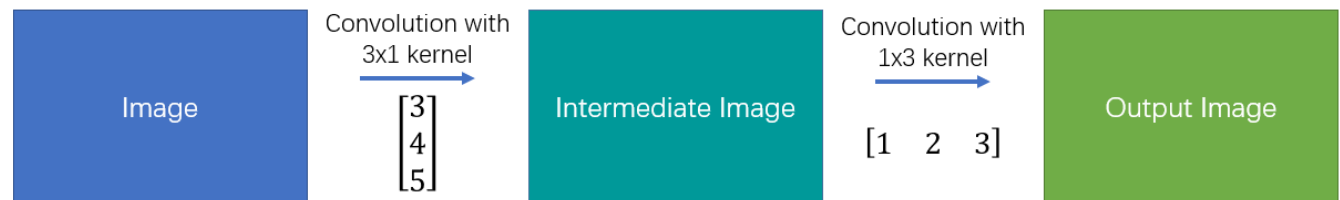
- Instead of doing one convolution with 9 multiplications, we do two convolutions with 3 multiplications each (6 in total) to **achieve the same effect**.
- With **less multiplications**, computational complexity goes down, and the network is able to run faster.

### Simple Convolution

$$\begin{bmatrix} 3 & 6 & 9 \\ 4 & 8 & 12 \\ 5 & 10 & 15 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$



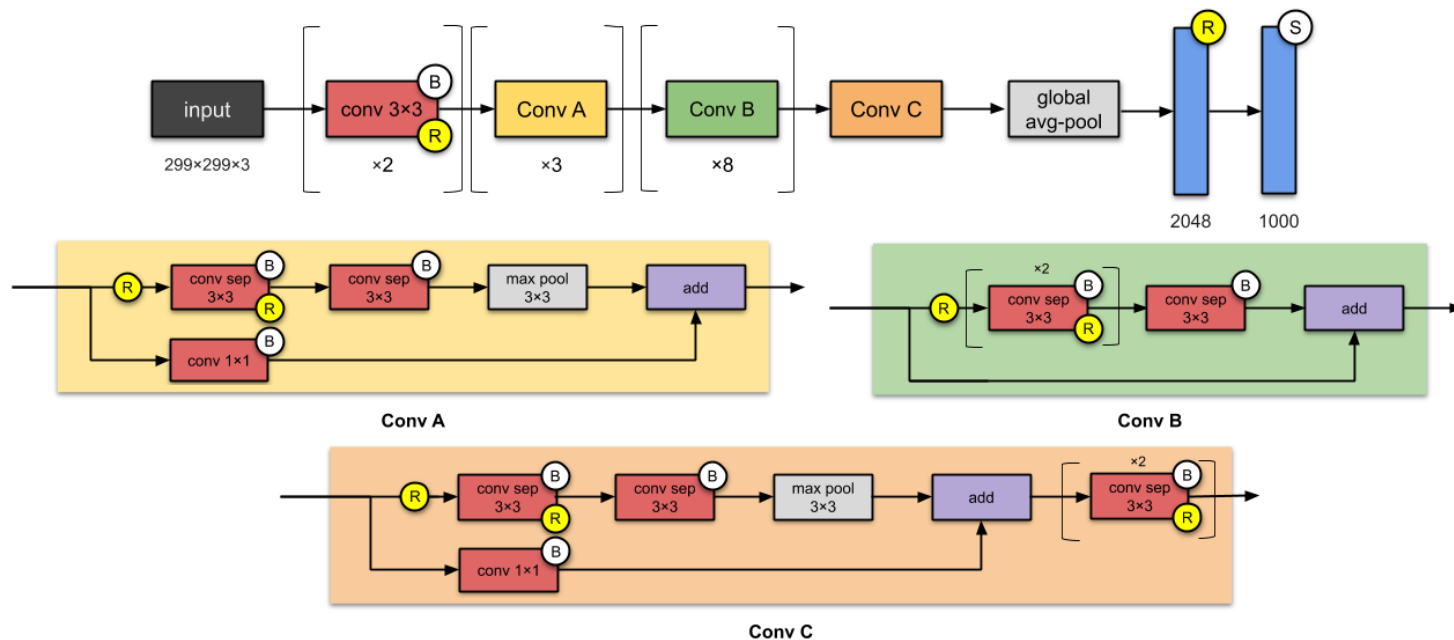
### Spatial Separable Convolution



# 3. Different models for CNN

## □ Xception (2016)

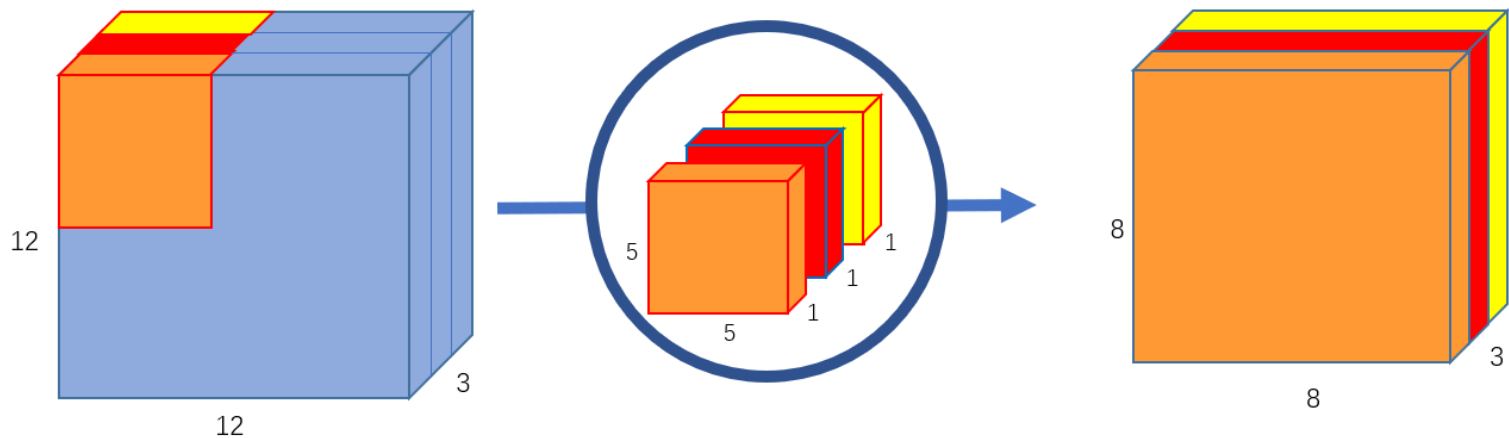
- invented by **Google**.
- is an adaptation from Inception.
- The Inception modules have been replaced with **depthwise separable convolutions**.
- has about 23M parameters.



# 3. Different models for CNN

## □ Depthwise separable convolutions

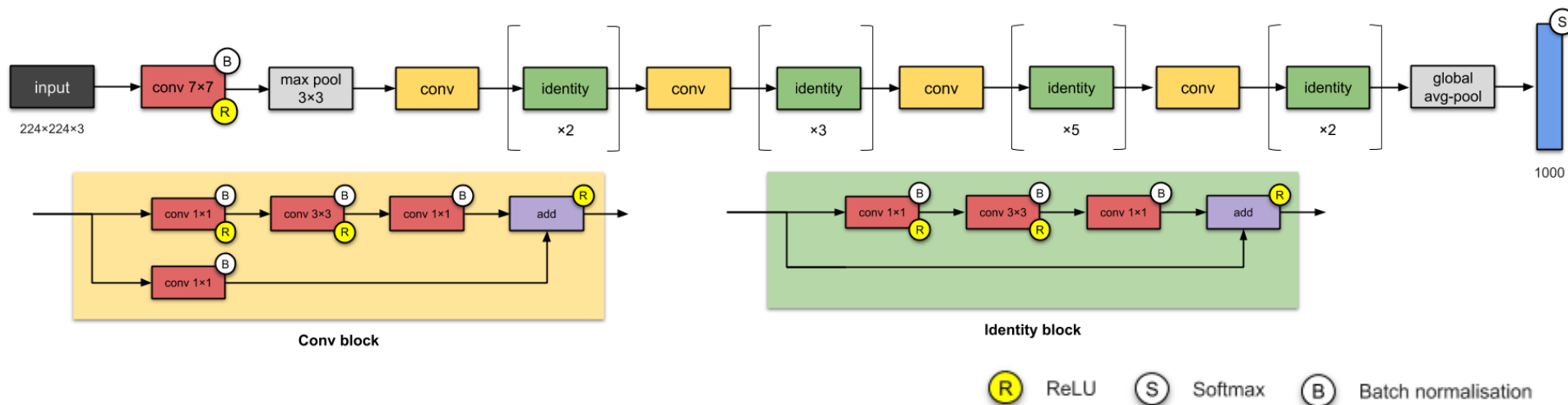
- The depthwise separable convolution is so named because it deals not just with the spatial dimensions, but with the depth dimension.
- Can have **different kernel matrix for different channels**.
- Good for the colour image, which usually has 3 channels, i.e. RGB.
- i.e. Depthwise convolution, uses 3 kernels to transform a  $12 \times 12 \times 3$  image to a  $8 \times 8 \times 3$  image.



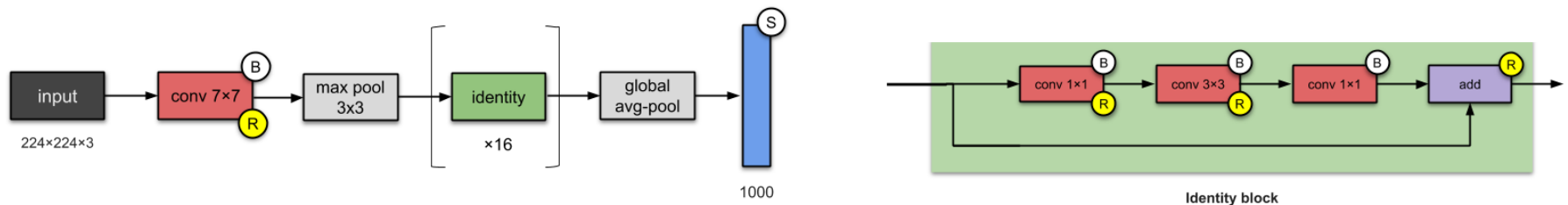
# 3. Different models for CNN

## □ ResNet-50 (2015)

- invented by **Microsoft Research**.
- is one of the early adopters of **batch normalisation**.
- has about 25.5M parameters.



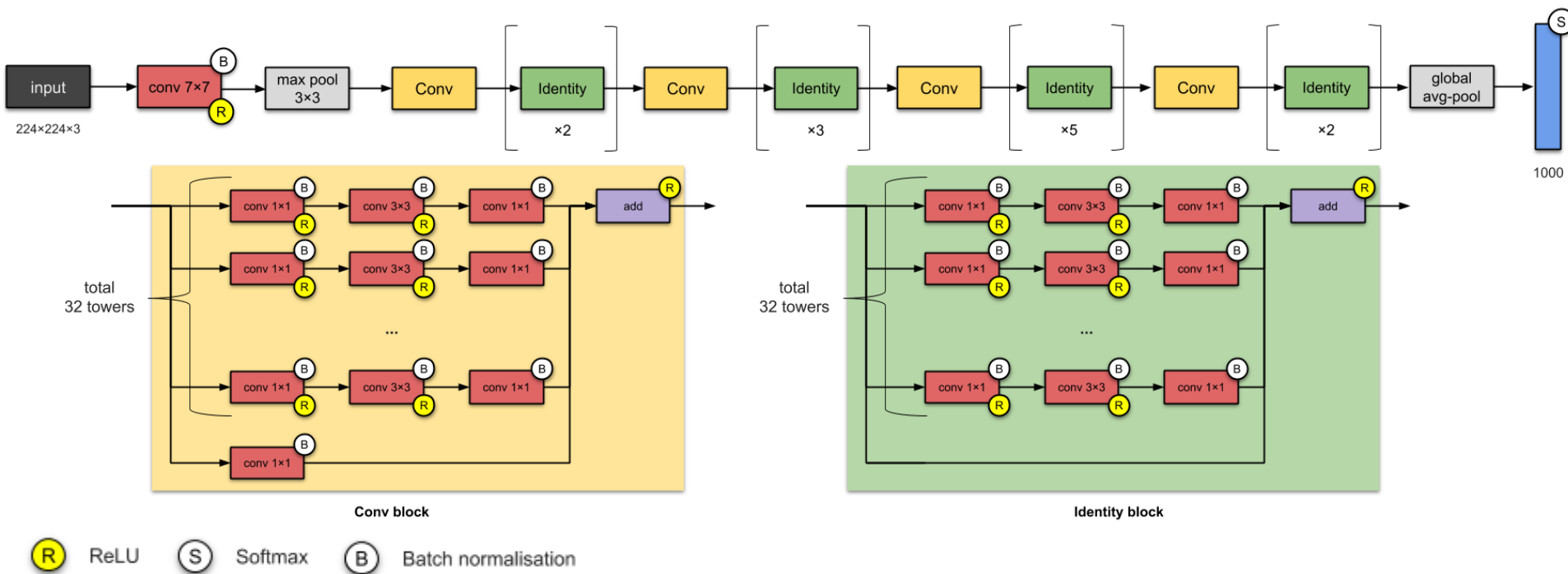
➔ This architecture can be **simplified as**.



# 3. Different models for CNN

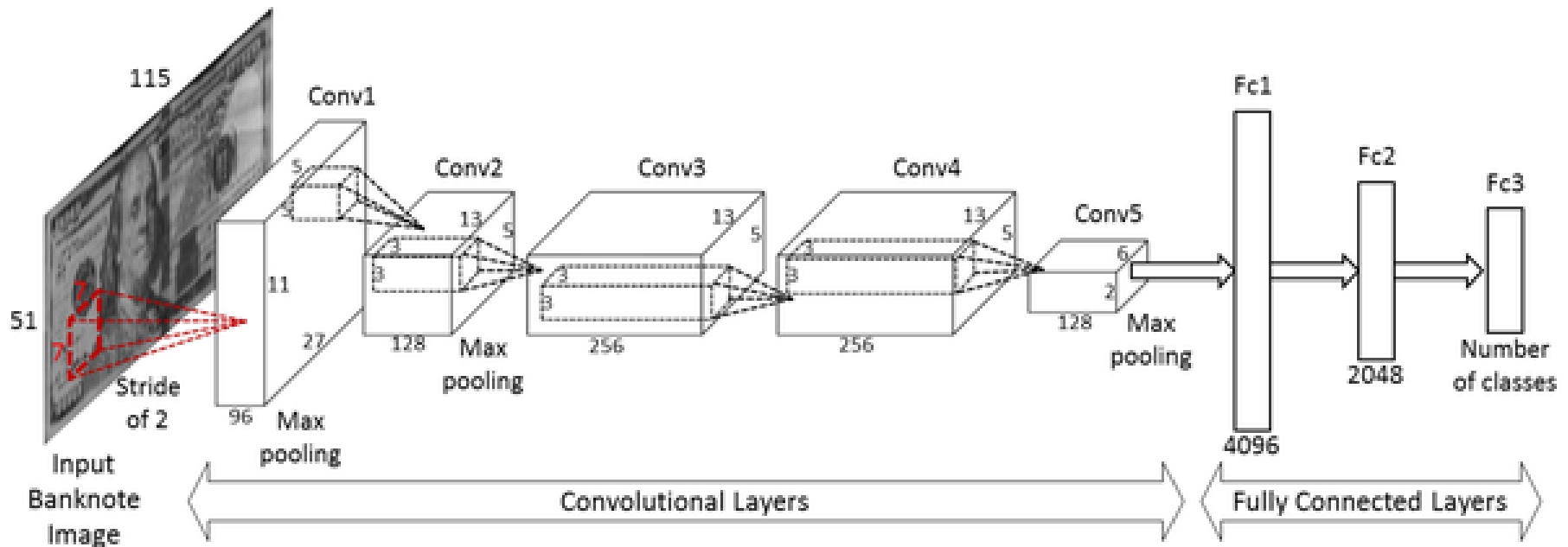
## □ ResNeXt-50 (2017)

- invented by **Facebook** Research and University of California San Diego.
- **adding of parallel towers/branches/paths** within each module to ResNet-50.
- has about 25M parameters.



## 4. For your project...

### □ Models

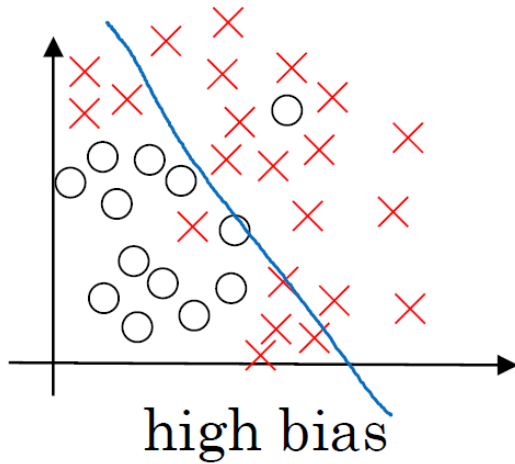


1) Select candidate models, then

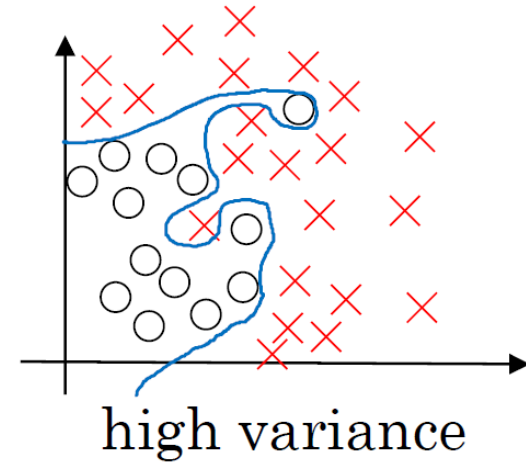
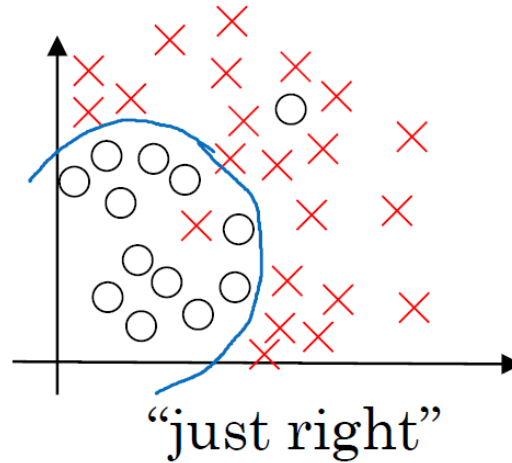
2) Test the selected models with different parameter setups if needed.

# 5. Why does mine not work?

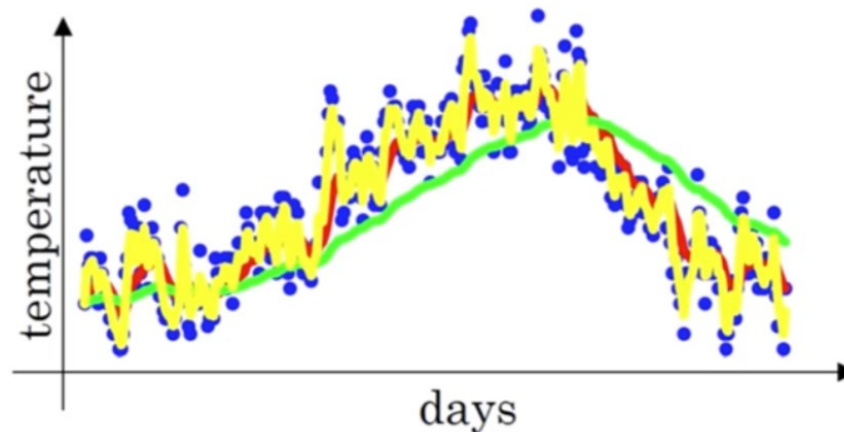
## □ Bias and Variance



**Underfitting**



**Overfitting**



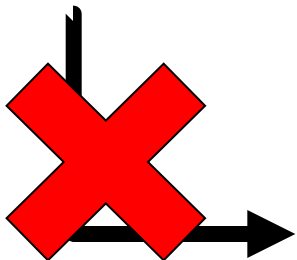
# 5. Why does mine not work?

## □ Face Detection

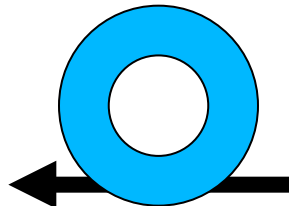


**Database**

**Model**



**Can detect  
face?**

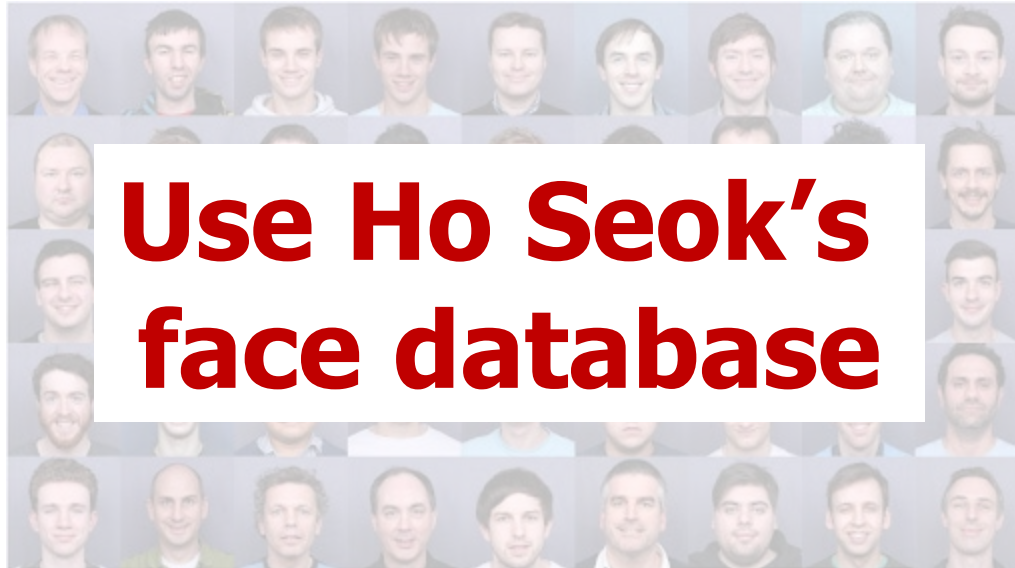


**Can detect  
face?**



## 5. Why does mine not work?

### ☐ Face Recognition



**Database** does not have  
Ho Seok's face

**Model**

